# Problem solving and search

## Chapter 3

# Outline

- Problem-solving agents
- Problem types
- Problem formulation
- Example problems
- Basic search algorithms

# Problem-solving agents

Restricted form of general agent:

```
function SIMPLE-PROBLEM-SOLVING-AGENT( percept) returns an action
  static: seq, an action sequence, initially empty
          state, some description of the current world state
          goal, a goal, initially null
          problem, a problem formulation

  state ← UPDATE-STATE(state, percept)
  if seq is empty then
      goal ← FORMULATE-GOAL(state)
      problem ← FORMULATE-PROBLEM(state, goal)
      seq ← SEARCH( problem)
  action ← RECOMMENDATION(seq, state)
  seq ← REMAINDER(seq, state)
  return action
```

# Example: Romania

On holiday in Romania; currently in Arad.
Flight leaves tomorrow from Bucharest

Formulate goal:

be in Bucharest

Formulate problem:

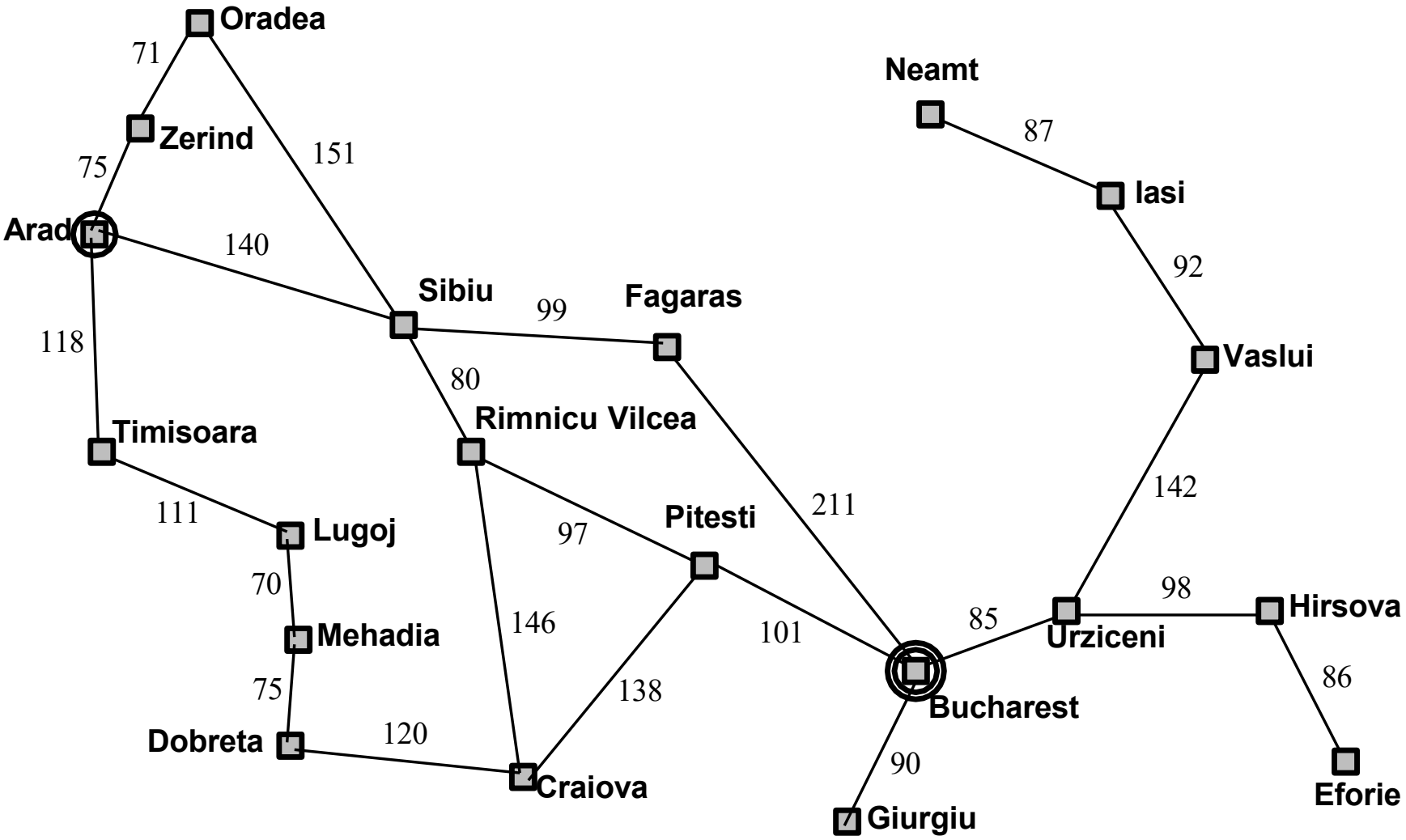
states: various cities

actions: drive between cities

Find solution:

sequence of cities, e.g., Arad, Sibiu, Fagaras, Bucharest

# Example: Romania



# Problem types

Deterministic, fully observable ==⇒ single-state problem

Agent knows exactly which state it will be in; solution is a sequence

Non-observable ==⇒ conformant problem

Agent may have no idea where it is; solution (if any) is a sequence

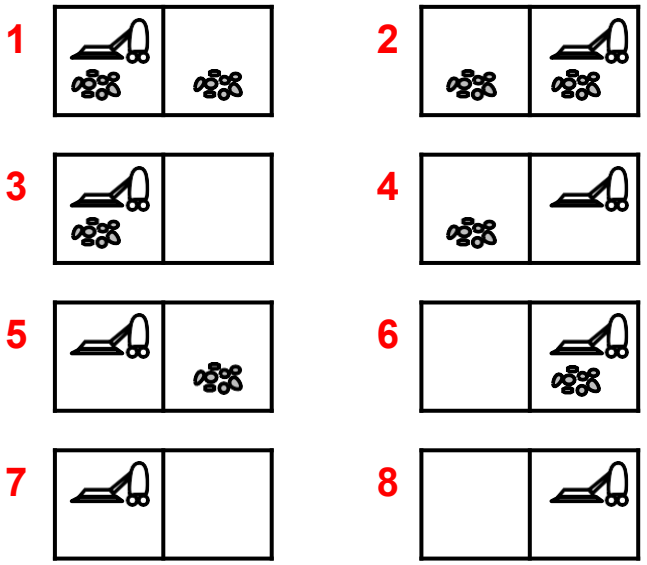
Nondeterministic and/or partially observable ==⇒ contingency problem percepts provide new information about current state

solution is a contingent plan or a policy
often interleave search, execution

Unknown state space ==⇒ exploration problem ("online")

# Example: vacuum world

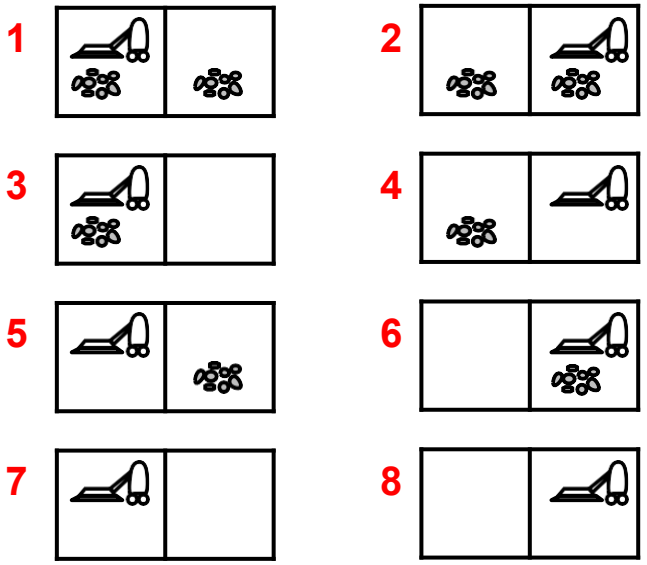
Single-state, start in #5. Solution??



# Example: vacuum world

Single-state, start in #5. Solution??
[*Right, Suck*]

Conformant, start in {1, 2, 3, 4, 5, 6, 7, 8}
e.g., *Right* goes to {2, 4, 6, 8}. Solution??

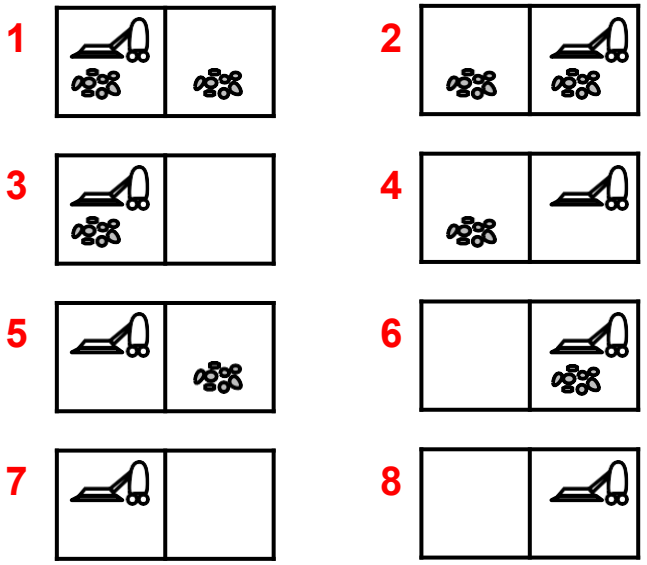


# Example: vacuum world

Single-state, start in #5. Solution??
[*Right, Suck*]

Conformant, start in {1, 2, 3, 4, 5, 6, 7, 8}
e.g., *Right* goes to {2, 4, 6, 8}. Solution??
[*Right, Suck, Left, Suck*]

Contingency, start in #5
Murphy's Law: *Suck* can dirty a clean carpet Local sensing: dirt, location only.
Solution??

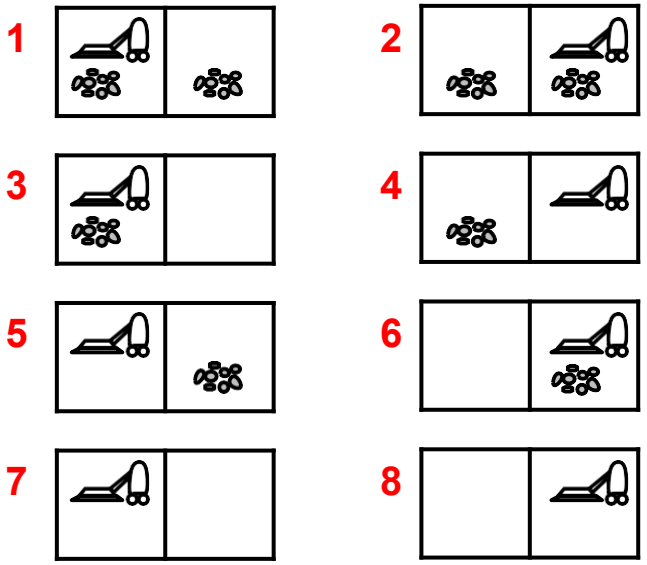


# Example: vacuum world

Single-state, start in #5. Solution??
[*Right, Suck*]

Conformant, start in {1, 2, 3, 4, 5, 6, 7, 8}
e.g., *Right* goes to {2, 4, 6, 8}.
Solution??
[*Right, Suck, Left, Suck*]

Contingency, start in #5
Murphy's Law: *Suck* can dirty a clean carpet Local sensing: dirt, location only.
Solution??
[*Right,* if *dirt* then *Suck*]



# Single-state problem formulation

A problem is defined by four items:

initial state e.g., "at Arad"

successor function $S(x)$ = set of action–state pairs
e.g., $S(Arad) = \{\langle Arad \rightarrow Zerind, Zerind\rangle, \ldots\}$

goal test, can be
explicit, e.g., $x$ = "at Bucharest" implicit,
e.g., $NoDirt(x)$

path cost (additive)
e.g., sum of distances, number of actions executed, etc.
$c(x, a, y)$ is the step cost, assumed to be $\geq 0$

A solution is a sequence of actions
leading from the initial state to a goal state

# Selecting a state space

Real world is absurdly complex
⇒ state space must be abstracted for problem solving

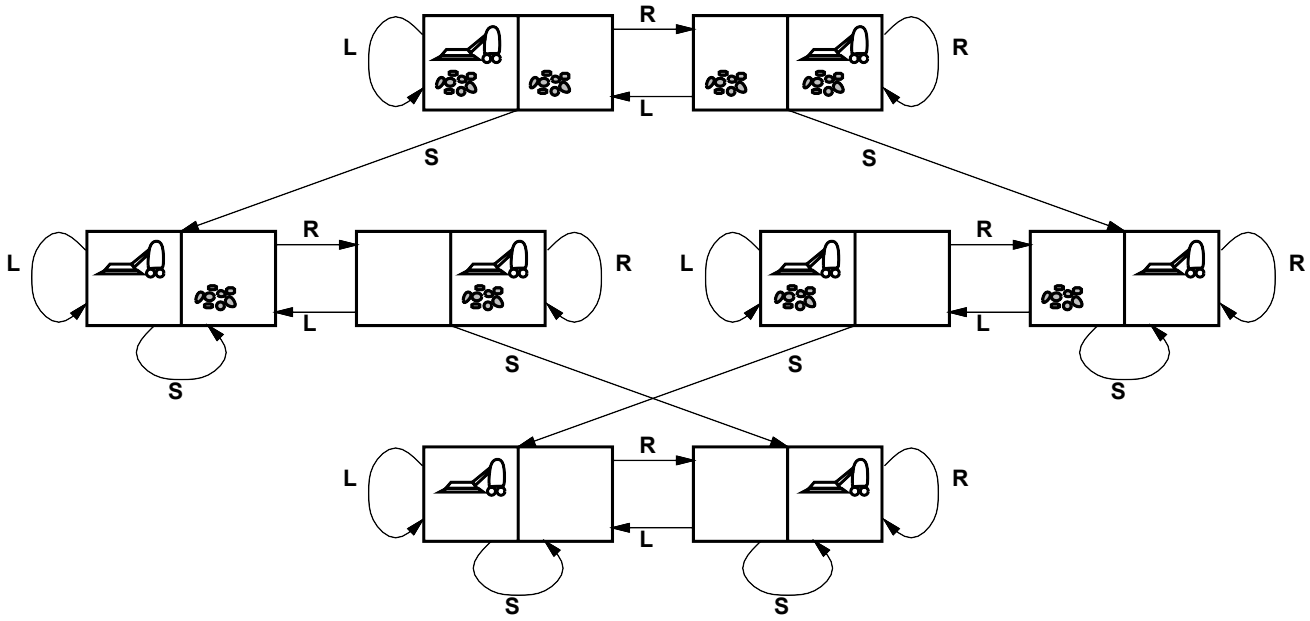
(Abstract) state = set of real states

(Abstract) action = complex combination of real actions
e.g., "Arad → Zerind" represents a complex set
of possible routes, detours, rest stops, etc.
For guaranteed realizability, any real state "in Arad"
must get to some real state "in Zerind"

(Abstract) solution =
set of real paths that are solutions in the real world

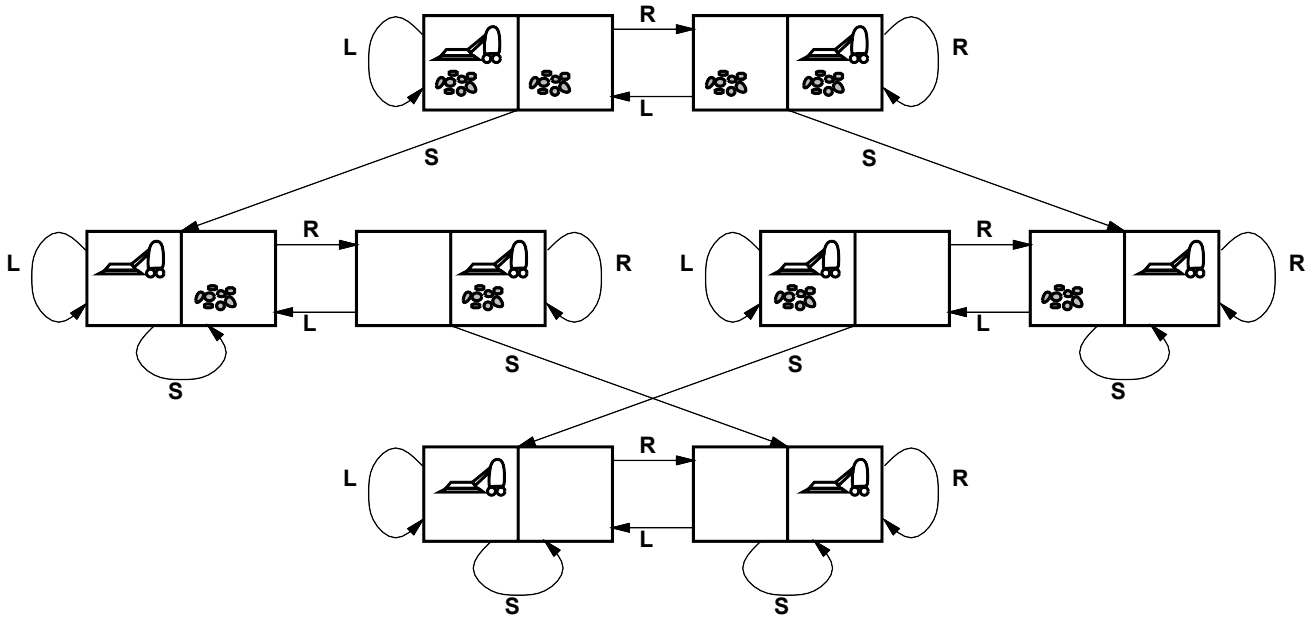
Each abstract action should be "easier" than the original problem!

# Example: vacuum world state space graph



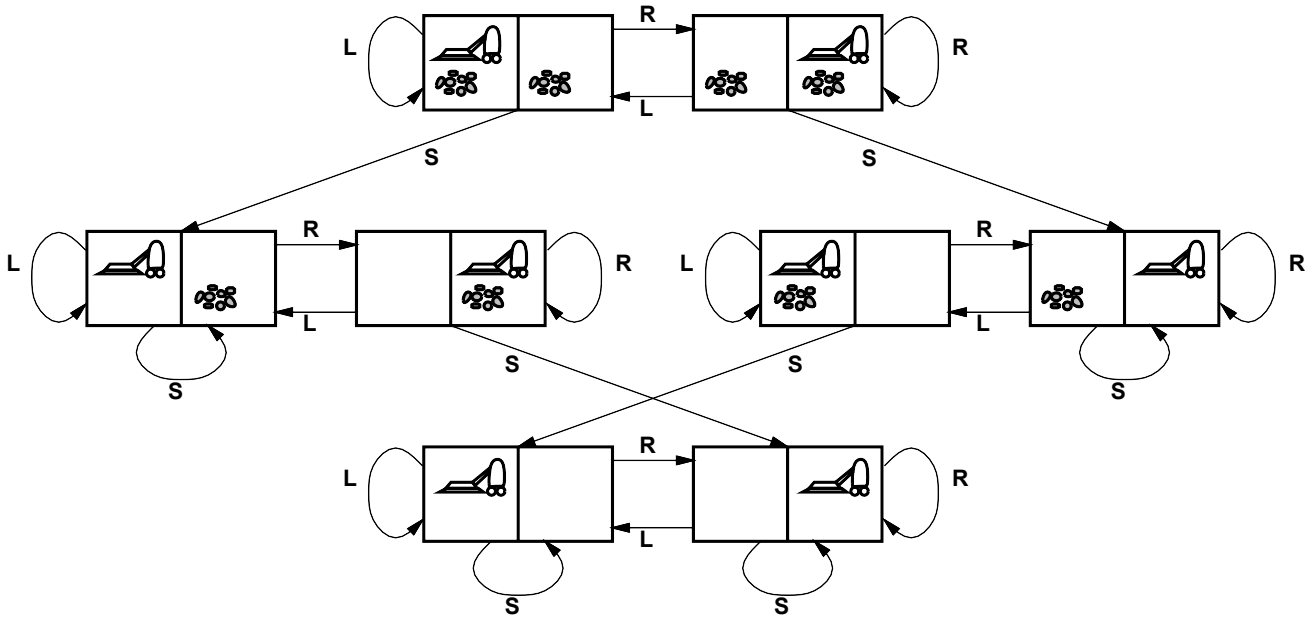

states??
actions??
goal test??
path cost??

# Example: vacuum world state space graph



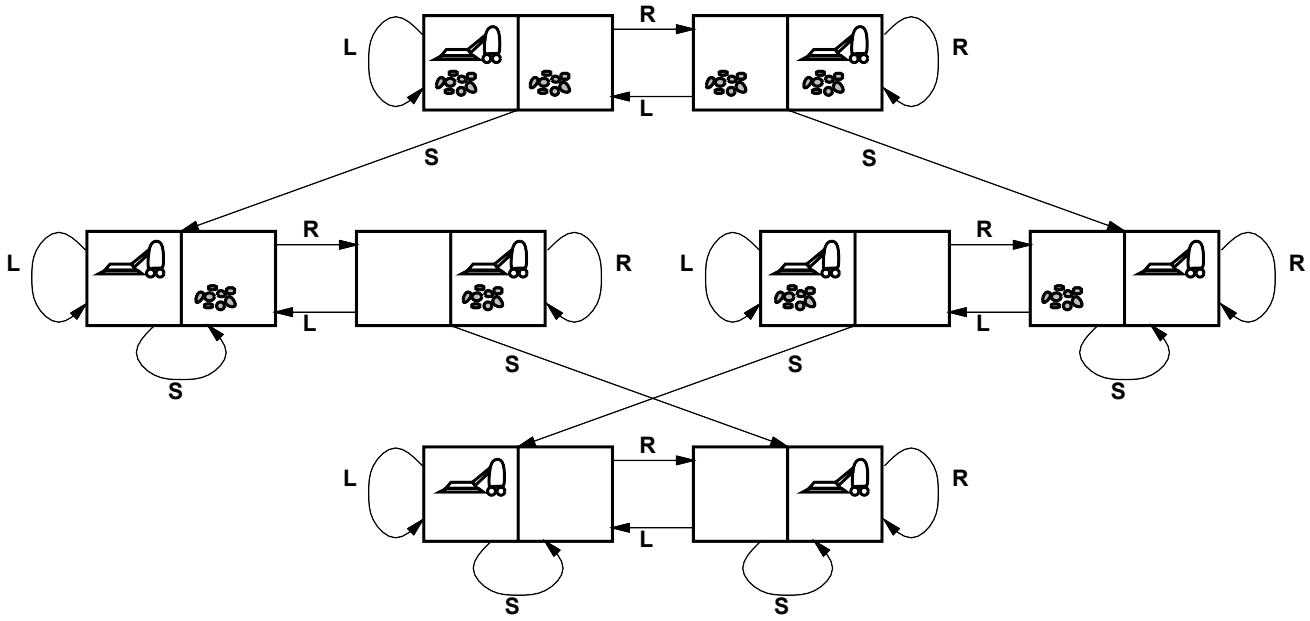

states??: integer dirt and robot locations (ignore dirt amounts etc.)
actions??
goal test??
path cost??

# Example: vacuum world state space graph



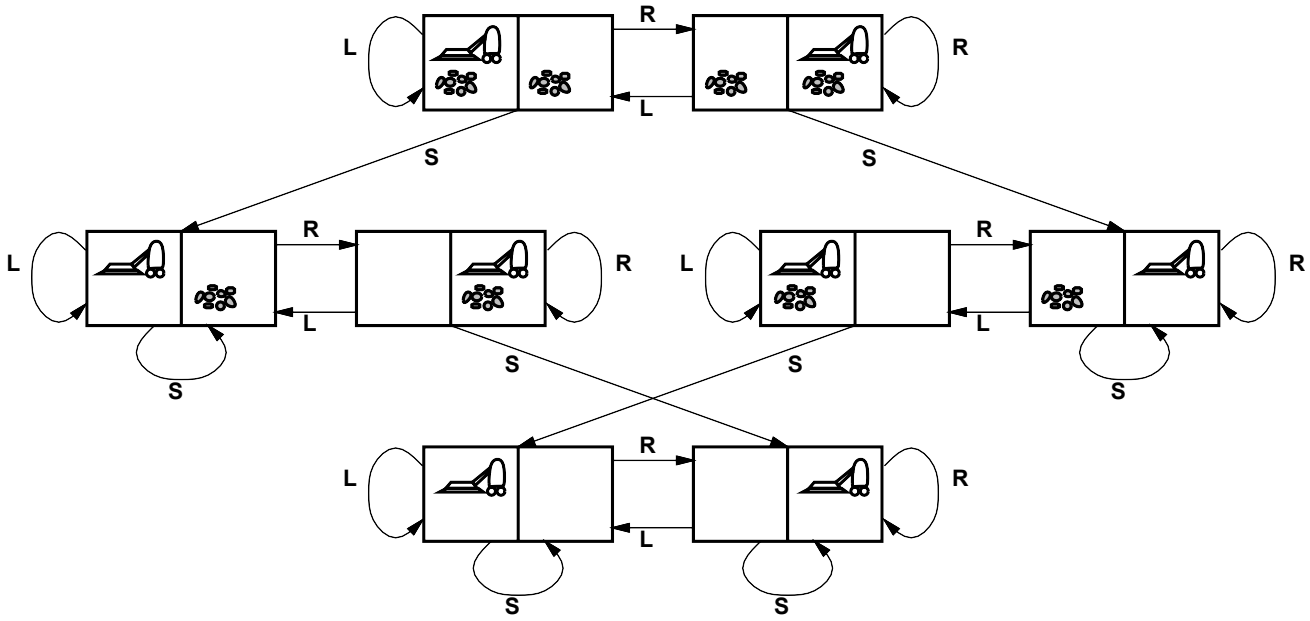

states??: integer dirt and robot locations (ignore dirt amounts etc.)
actions??: *Left*, *Right*, *Suck*, *NoOp*
goal test??
path cost??

# Example: vacuum world state space graph



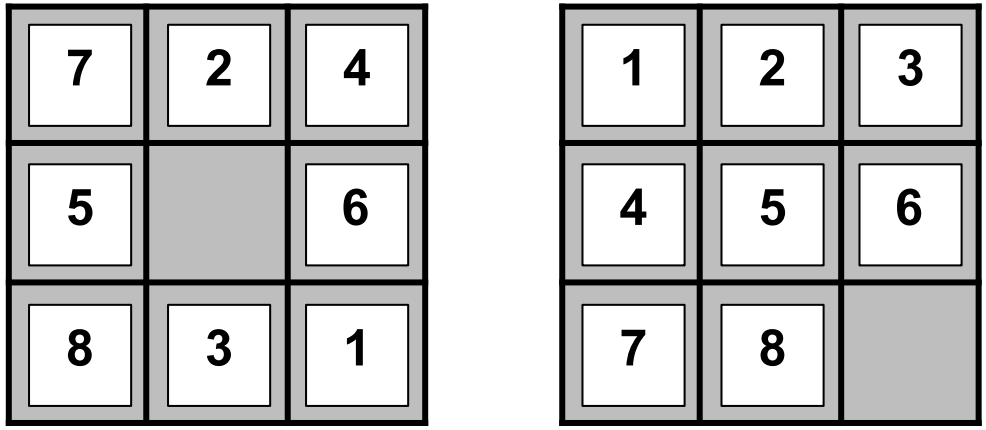

states??: integer dirt and robot locations (ignore dirt amounts etc.)
actions??: *Left*, *Right*, *Suck*, *NoOp*
goal test??: no dirt
path cost??

# Example: vacuum world state space graph




states??: integer dirt and robot locations (ignore dirt amounts etc.)
actions??: *Left*, *Right*, *Suck*, *NoOp*
goal test??: no dirt
path cost??: 1 per action (0 for *NoOp*)

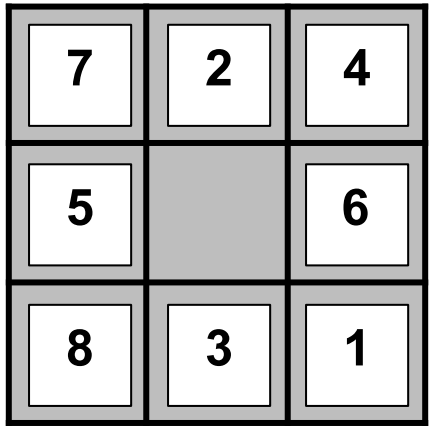
# Example: The 8-puzzle



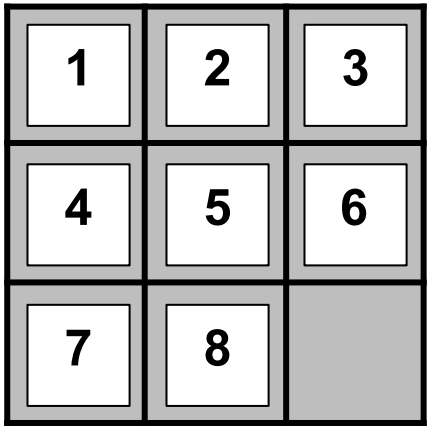

**Start State** **Goal State**

states??
actions??
goal test??
path cost??

# Example: The 8-puzzle



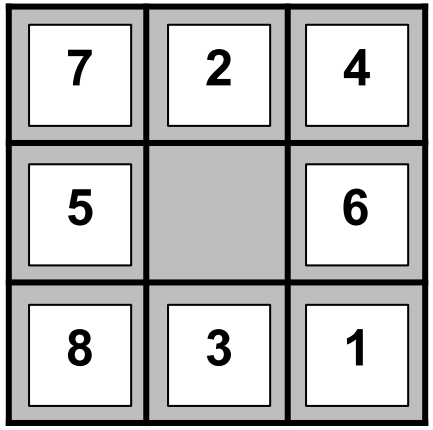

Start State



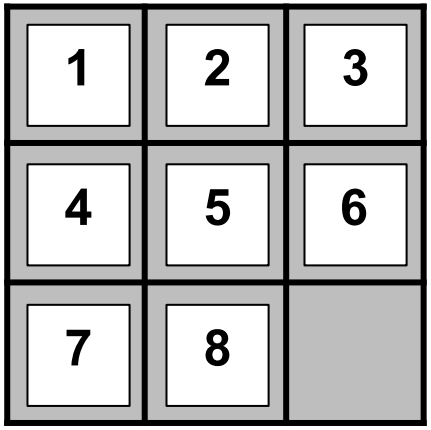

Goal State

states??: integer locations of tiles (ignore intermediate positions)
actions??
goal test??
path cost??

# Example: The 8-puzzle



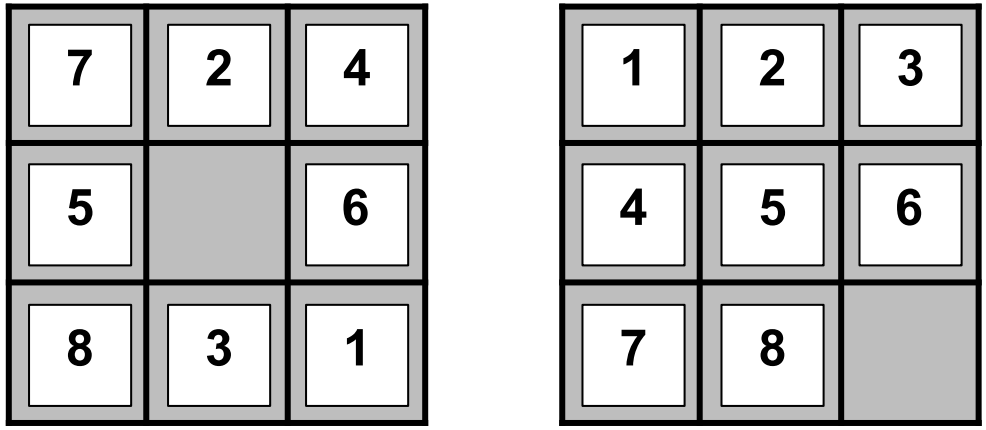

Start State




Goal State

states??: integer locations of tiles (ignore intermediate positions)
actions??: move blank left, right, up, down (ignore unjamming etc.)
goal test??
path cost??

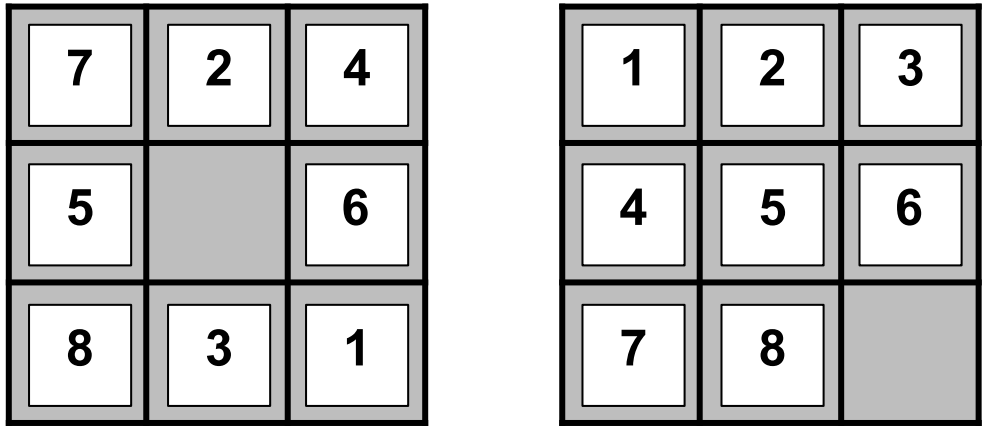
# Example: The 8-puzzle




Start State  Goal State

states??: integer locations of tiles (ignore intermediate positions)
actions??: move blank left, right, up, down (ignore unjamming etc.)
goal test??: = goal state (given)
path cost??

# Example: The 8-puzzle

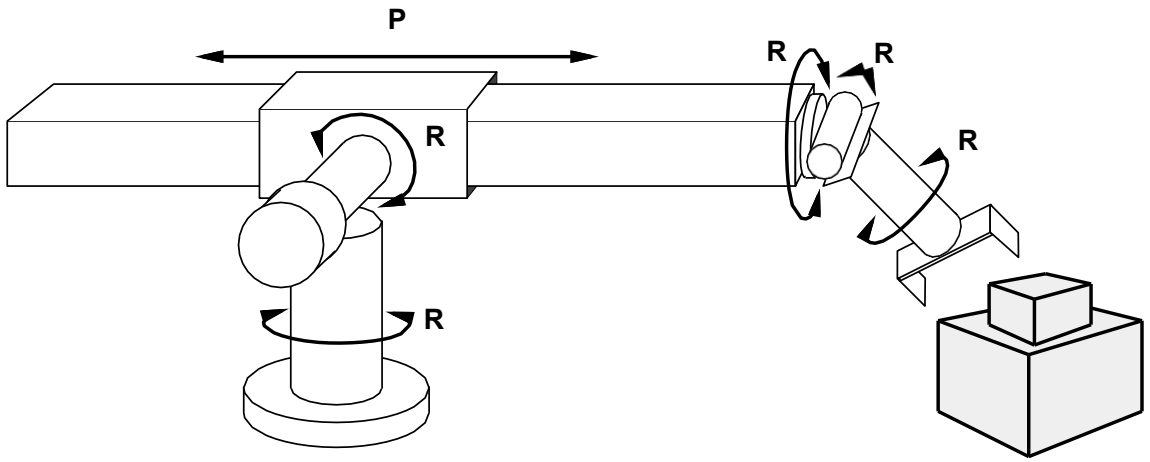



Start State  Goal State

states??: integer locations of tiles (ignore intermediate positions)
actions??: move blank left, right, up, down (ignore unjamming etc.)
goal test??: = goal state (given)
path cost??: 1 per move

[Note: optimal solution of *n*-Puzzle family is NP-hard]

# Example: robotic assembly




states??: real-valued coordinates of robot joint angles parts of the object to be assembled

actions??: continuous motions of robot joints

goal test??: complete assembly with no robot included!

path cost??: time to execute